

ELE216

Développement de logiciels en génie électrique

Pascal Giard

Département de génie électrique

Cours 3: Programmation sous GNU/Linux:
Programmes et processus, gestion de mémoire, et déverminage

PROGRAMMES ET PROCESSUS

Programmes et processus

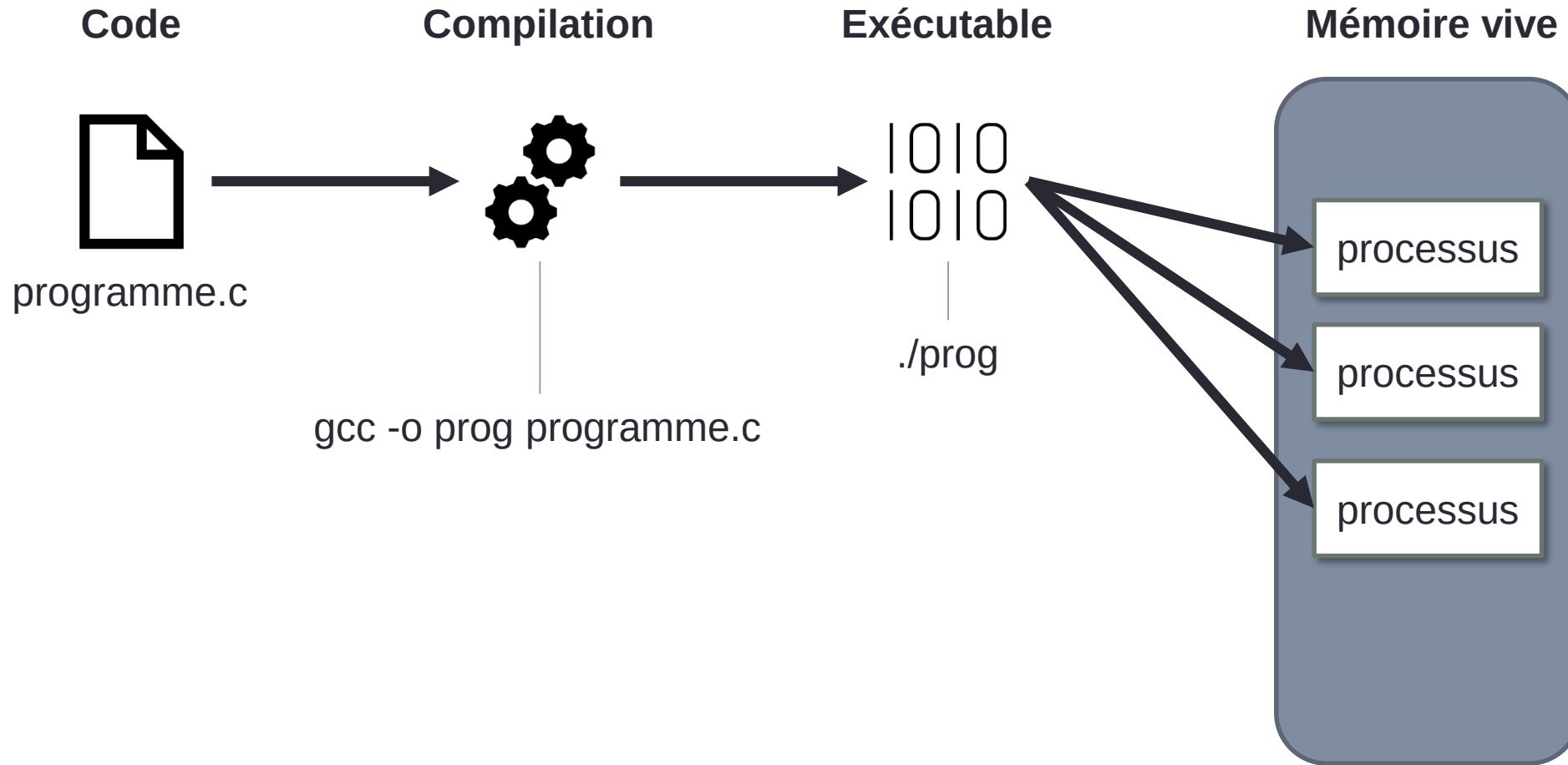
❑ Définition d'un programme

- ❖ Un programme est une séquence d'instructions à être exécutées par un processeur
- ❖ Un programme est généralement obtenu en compilant du code source.

❑ Définition d'un processus

- ❖ Un processus peut être vu comme une instance d'un programme qui est chargé en mémoire et en cours d'exécution.

Programmes et processus



Le bloc de contrôle d'un processus et ses attributs

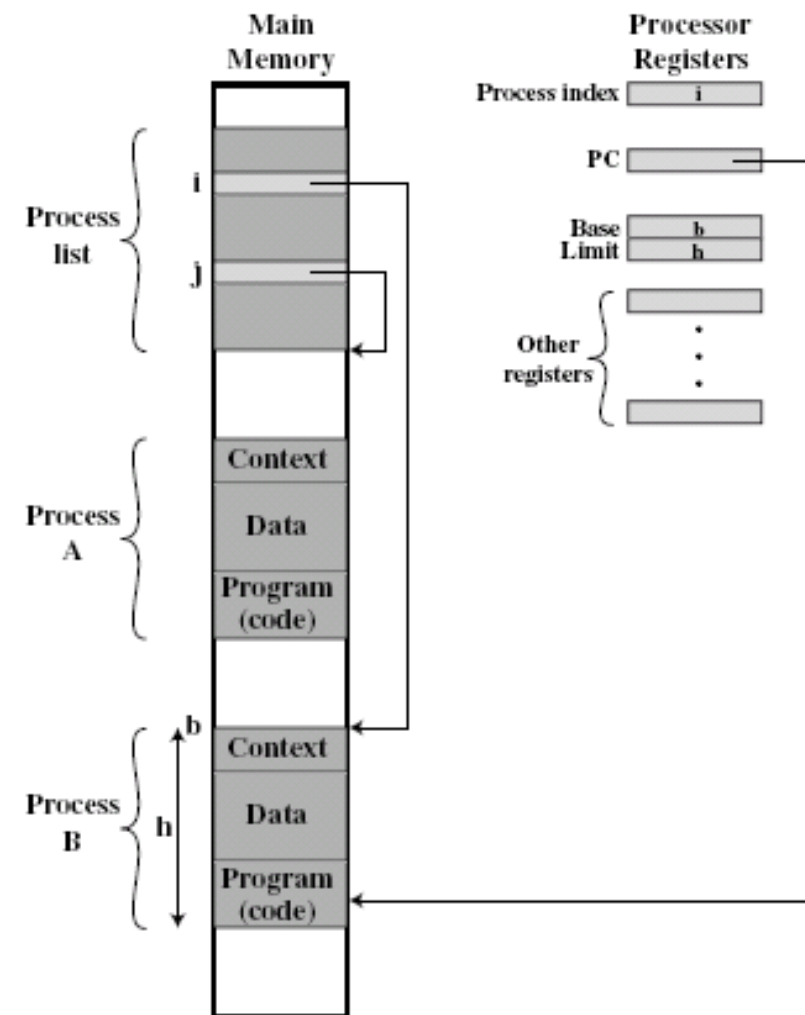
- ☐ État
- ☐ Identifiant (numéro unique)
- ☐ Valeurs des registres et compteur ordinal
- ☐ Nom de l'utilisateur propriétaire du processus
- ☐ Priorité (pour l'ordonnancement)
- ☐ Informations sur l'espace d'adressage mémoire
- ☐ Informations sur les E/S effectuées (descripteurs de fichiers ouverts, répertoire courant, etc.)
- ☐ Informations sur les ressources utilisées (user time, real time, etc.)



Process Control Block (PCB)

Processus concurrents

- ❑ Dans un environnement concurrent, plusieurs processus peuvent coexister:
 - ❖ Les processus se partagent le(s) processeur(s)
 - ❖ Des processus différents peuvent exécuter le même programme
 - ❖ Chaque processus se voit assigner une plage mémoire distincte
 - ❖ Un processus n'a pas directement accès aux plages mémoires des autres processus
- ❑ Linux partage les ressources équitablement entre les processus



L'information sur les processus dans GNU/Linux

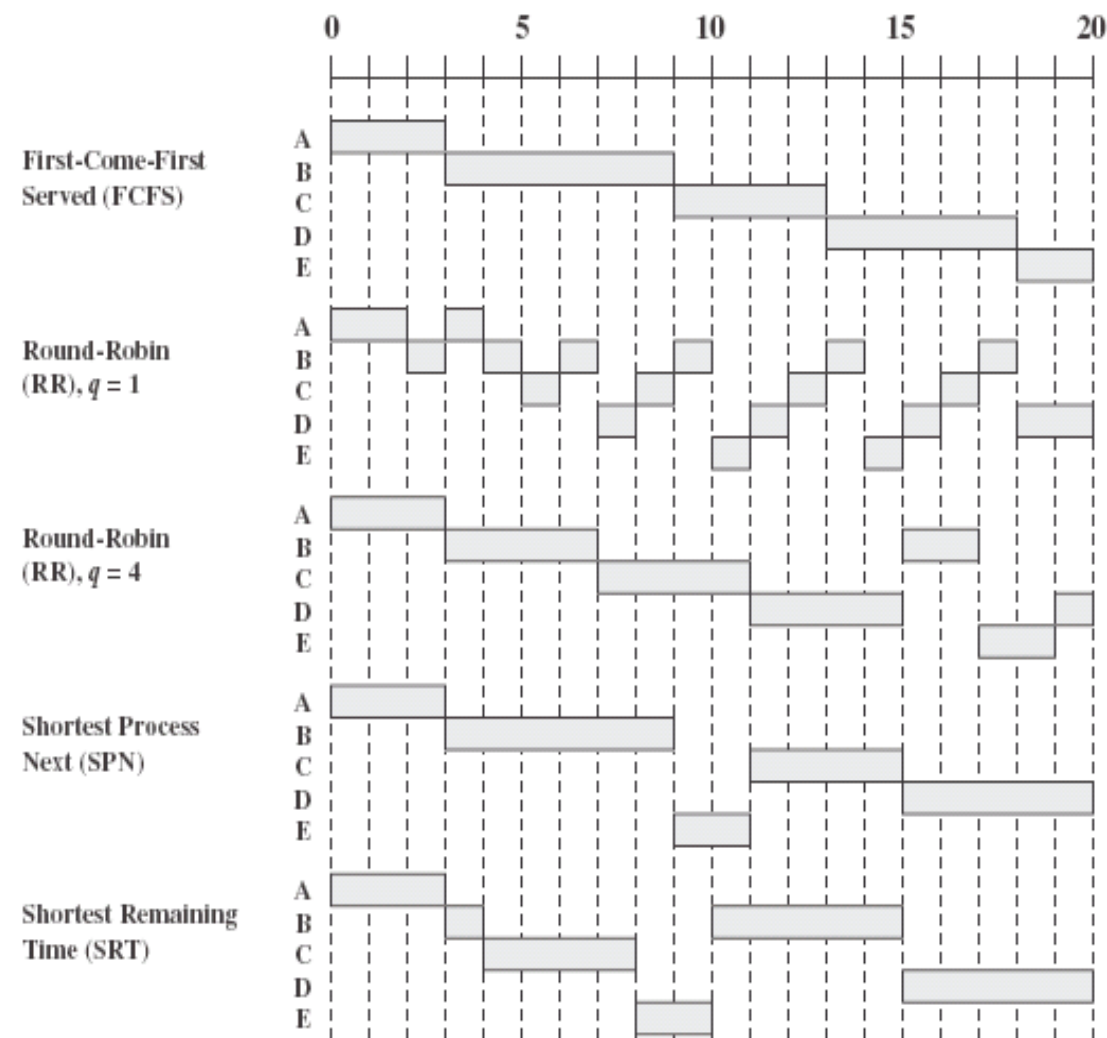
- ❑ La commande “ps” permet de voir la liste des processus actuellement vivants
- ❑ Le dossier /proc contient l'information relative aux différents processus

Exemple avec la commande “ps”

- ❑ Ouvrez un terminal et créez le répertoire “ELE216”. (\$ mkdir ELE216)
- ❑ Positionnez-vous dans ce nouveau répertoire. (\$ cd ELE216)
- ❑ Récupérez le code du projet “cours03-ex” sur GitLab. (\$ git clone <lien gitlab>)
- ❑ Compilez le programme “getchar.c” situé dans le répertoire “exemples”.
 - ❖ \$ gcc -g -o getchar getchar.c
- ❑ Exécutez le programme getchar (\$./getchar)
- ❑ Alors que le programme getchar roule encore, ouvrez une nouvelle fenêtre de terminal
- ❑ Entrez la commande: \$ ps x | grep getchar
- ❑ Le premier chiffre est le PID du processus où roule le programme getchar

Ordonnancement des processus

- ❑ Le noyau (kernel) est responsable de déterminer quel processus s'exécute quand et où
- ❑ Il existe plusieurs algorithmes d'ordonnancement
- ❑ Chaque changement de processus a un coût en termes de performance



Exemples d'algos et de politiques

GESTION DE MÉMOIRE

Allocation statique et dynamique de mémoire

Espace d'adressage d'un processus

- ❑ Chaque processus reçoit un espace d'adressage en mémoire vive qui lui est alloué. Cet espace d'adressage contient:
 - ❖ Les instructions du programme
 - ❖ Les données du processus, qui sont décomposées en deux segments:
 - Le segment **data** (variables globales initialisées et les constantes)
 - Le segment **bss** (variables globales non initialisées)
 - ❖ La pile (stack)
 - ❖ Le tas (heap)
 - ❖ Les instructions et les données des bibliothèques partagées utilisées par le processus

Espace d'adressage

❑ La pile (stack). La pile contient les données suivantes:

- ❖ Les variables automatiques d'un programme
- ❖ Les paramètres des fonctions
- ❖ La valeur de retour des fonctions
- ❖ L'adresse de retour des fonctions
- ❖ De l'information qui doit être sauvegardée avant d'exécuter une fonction.

```

111 // output the house status to stdin
112 void list_house(house_t* house) {
113     int i;
114     printf("\n");
115     printf("La piece par défaut est: %s\n\n",
116           get_default_room_name(house));
117     printf("Les pièces de la maison sont:\n");
118     for (i = 0; i < house->nb_rooms; i++) {
119         list_room(house->rooms[i]);
120     }
121     printf("\n");
122 }
  
```

```

56 // return the room name
57 char* get_room_name(room_t* room) {
58     return room->room_name;
59 }
  
```

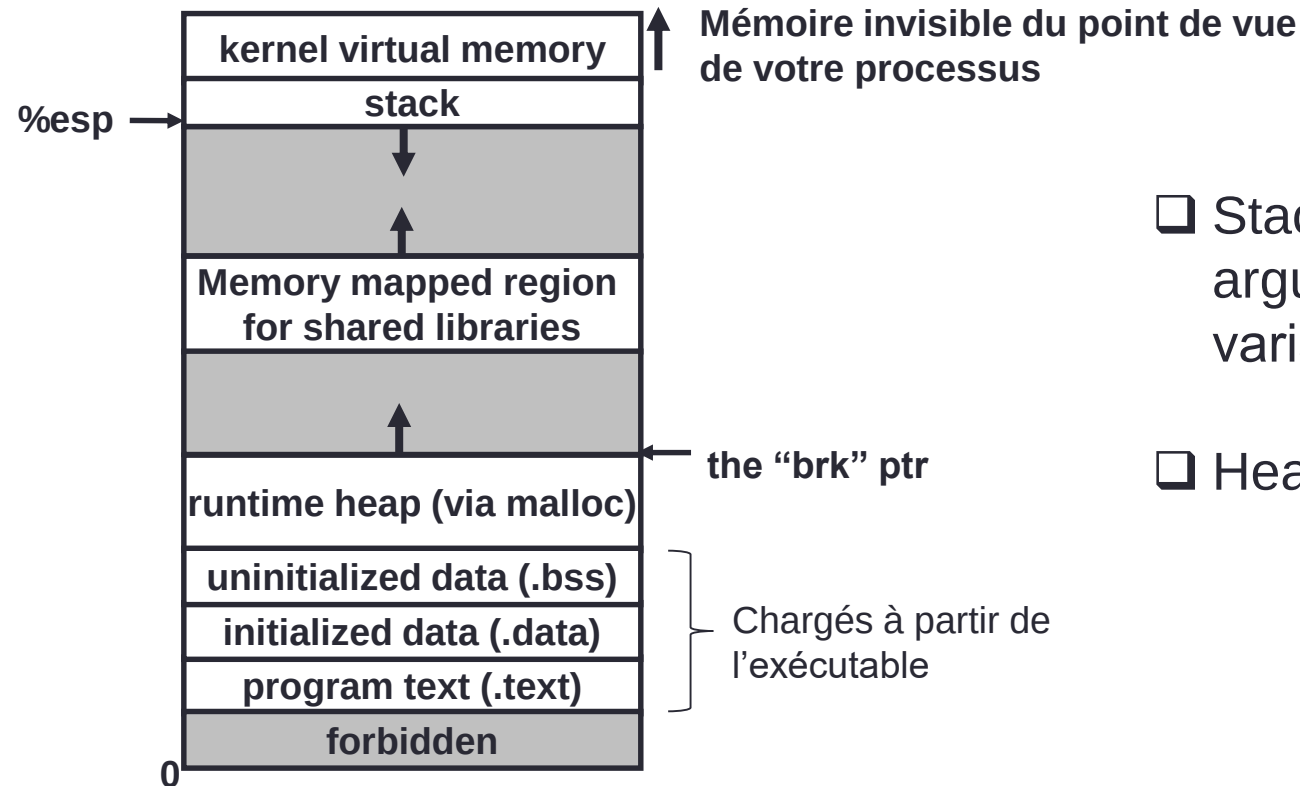
```

80 // output the room status to stdin
81 void list_room(room_t* room) {
82     int i;
83
84     printf("%30s - états des appareils: ", room->room_name);
85     for (i = 0; i < room->nb_appliance; i++) {
86         printf("%d ", room->appliance_state[i]);
87     }
88     printf("\n");
89 }
  
```

Espace d'adressage

- ❑ Le tas (heap). Le tas contient:
 - ❖ La mémoire allouée dynamiquement avec les instructions malloc(), calloc(), realloc(), etc.
 - ❖ Cette mémoire peut être récupérée avec l'instruction free().

Espace d'adressage d'un processus



- ❑ Stack: variables locales, y compris les arguments de la ligne de commande et les variables d'environnement
- ❑ Heap: mémoire dynamique

Régions de mémoire

- ❑ L'espace d'adressage d'un processus est composé de plusieurs régions de mémoire.
- ❑ On peut visualiser les régions de mémoire associées à un processus via le répertoire `/proc`
- ❑ Compilez le programme « `stockage.c` » et exécutez-le.

Différents types de stockage en mémoire

```

1  /*
2   * Titre: stockage.c
3   * Auteur: Vincent Lacasse
4   * Date: 2021-09-16
5   *
6   * Programme qui démontre l'organisation du stockage
7   * dans un processus Linux
8   */
9
10 #include <stdio.h>
11 #include <stdlib.h>
12 #include <sys/types.h>
13 #include <unistd.h>
14
15 int global; /* variable globale */
16
17 int main() {
18     char* constante = "Ceci est une constante";
19     int local; /* variable locale */
20     int* heap; /* utilisation du heap */
21     heap = malloc(sizeof(int));
22
23     printf("Numero du processus: %d\n", getpid());
24     printf("Adresse de constante: %p\n", (void*)&constante);
25     printf("Adresse de la zone memoire pointee par constante
26           %p\n", (void*)constante);
27     printf("Adresse de global: %p\n", (void*)&global);
28     printf("Adresse de local: %p\n", (void*)&local);
29     printf("Adresse de heap: %p\n", (void*)&heap);
30     printf("Adresse de la zone memoire pointee par heap: %p\n",
31           (void*)heap);
32
33     free(heap);
34     getchar(); /* Juste pour ne pas quitter */
35     return 0;
36 }
  
```

```

vlacasse@ele-a25502-00: ~/ELE216/cours03-code/exemples
Fichier Édition Affichage Rechercher Terminal Aide
vlacasse@ele-a25502-00:~/ELE216/cours03-code/exemples$ gcc -g -o stockage stockage.c
vlacasse@ele-a25502-00:~/ELE216/cours03-code/exemples$ ./stockage
Numero du processus: 1784
Adresse de constante: 0x7ffd745b7518
Adresse de la zone memoire pointee par constante 0x5638a7ef9968
Adresse de global: 0x5638a80fa014
Adresse de local: 0x7ffd745b7514
Adresse de heap: 0x7ffd745b7520
Adresse de la zone memoire pointee par heap: 0x5638a99a3260
  
```

Puis faites: \$ cat /proc/1784/maps
(remplacez 1784 par le PID que le programme stockage vous donne)

Différents types de stockage en mémoire

/proc/[PID]/maps

```
vlacasse@ele-a25502-00: ~/ELE216/cours03-code/exemples
Fichier  Édition  Affichage  Rechercher  Terminal  Aide
vlacasse@ele-a25502-00:~/ELE216/cours03-code/exemples$ cat /proc/1784/maps
5638a7ef9000-5638a7efa000 r-xp 00000000 00:33 32774153 /home/etsmtl/vlacasse/ELE216/cours03-code/exemples/stockage
5638a80f9000-5638a80fa000 r--p 00000000 00:33 32774153 /home/etsmtl/vlacasse/ELE216/cours03-code/exemples/stockage
5638a80fa000-5638a80fb000 rw-p 00001000 00:33 32774153 /home/etsmtl/vlacasse/ELE216/cours03-code/exemples/stockage
5638a99a3000-5638a99c4000 rw-p 00000000 00:00 0 [heap]
7f8a10610000-7f8a107f7000 r-xp 00000000 08:02 2359805 /lib/x86_64-linux-gnu/libc-2.27.so
7f8a107f7000-7f8a109f7000 ---p 001e7000 08:02 2359805 /lib/x86_64-linux-gnu/libc-2.27.so
7f8a109f7000-7f8a109fb000 r--p 001e7000 08:02 2359805 /lib/x86_64-linux-gnu/libc-2.27.so
7f8a109fb000-7f8a109fd000 rw-p 001eb000 08:02 2359805 /lib/x86_64-linux-gnu/libc-2.27.so
7f8a109fd000-7f8a10a01000 rw-p 00000000 00:00 0
7f8a10a01000-7f8a10a28000 r-xp 00000000 08:02 2359495 /lib/x86_64-linux-gnu/ld-2.27.so
7f8a10c07000-7f8a10c09000 rw-p 00000000 00:00 0
7f8a10c28000-7f8a10c29000 r--p 00027000 08:02 2359495 /lib/x86_64-linux-gnu/ld-2.27.so
7f8a10c29000-7f8a10c2a000 rw-p 00028000 08:02 2359495 /lib/x86_64-linux-gnu/ld-2.27.so
7f8a10c2a000-7f8a10c2b000 rw-p 00000000 00:00 0
7ffd74598000-7ffd745b9000 rw-p 00000000 00:00 0 [stack]
7ffd745c2000-7ffd745c5000 r--p 00000000 00:00 0 [vvar]
7ffd745c5000-7ffd745c7000 r-xp 00000000 00:00 0 [vdso]
ffffffff600000-ffffffff601000 r-xp 00000000 00:00 0 [vsyscall]
vlacasse@ele-a25502-00:~/ELE216/cours03-code/exemples$
```

Allocation dynamique (`malloc()` ou `calloc()`)

- ❑ On ne sait pas toujours à l'avance combien de mémoire sera nécessaire
 - ❖ Allocation dynamique sur le tas (heap)
- ❑ Pour des questions de performance, préférable d'allouer toute la mémoire nécessaire:
 - ❖ La mémoire contiguë mène à des lectures alignées, donc plus rapides
- ❑ Il est important de libérer la mémoire une fois qu'on a terminé

C'est quoi la différence entre `malloc()` et `calloc()` déjà?

Erreur fréquente: mémoire allouée inaccessible

- ❑ Attention de ne pas laisser de la mémoire allouée flotter sans pointeur pour y accéder.
- ❑ Exemple de situation où ça peut survenir:
 1. Écraser un pointeur vers un bloc existant
 2. Oublier de libérer un bloc, et réutiliser son pointeur
- ❑ Outil qui peut nous aider: valgrind

```
1  /*
2   * Titre: losttouch.c
3   * Auteur: Vincent Lacasse
4   * Date: 2021-09-17
5   *
6   * Exemple de mauvaise manipulation de pointeur qui
7   * rend un segment de mémoire allouée irrecupérable.
8   */
9
10 #include <stdio.h>
11 #include <stdlib.h>
12
13 int main() {
14     int* ptr;
15     ptr = (int*) malloc(5 * sizeof(int));
16     printf("Adresse de ptr: %p\n", (void *)ptr);
17
18     // La prochaine ligne de code est une erreur.
19     // Elle écrase la valeur de ptr ce qui rend le premier bloc
20     // de 5 entiers inaccessible et impossible à libérer avec free()
21
22     ptr = (int*) malloc(7 * sizeof(int));
23     printf("Adresse de ptr: %p\n", (void *)ptr);
24
25     // seul le bloc de 7 entiers est libéré
26     free(ptr);
27
28     // le bloc de 5 entiers ne peut pas être
29     // libéré car on ne connaît plus son adresse.
30     return 0;
31 }
32
```

Réallouer un segment de mémoire

- ❑ Pour augmenter la taille de notre segment déjà alloué avec `malloc()`
→ `realloc(dst, qté)`

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h> /* strcpy */

#define N 8

int main() {
    char* strptr;
    strptr = (char*)malloc(N*sizeof(char));

    strcpy(strptr, "Allo!");
    printf("%s\n", strptr);

    /* Oops, finalement on a changé d'idée et ce n'était pas assez d'espace! */
    strptr = (char*)realloc(strptr, 2*N*sizeof(char));
    strcat(strptr, " ELE216");
    printf("%s\n", strptr);

    free(strptr);

    return 0;
}
```

realloc.c

Remarque:
l'adresse du bloc peut
changer

Valeur ou adresse ?

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h> /* pour uint8_t */

#define N 5

void printA(const int *a, uint8_t n) {
    for(uint8_t i = 0; i < n; ++i) {
        printf("%d\t", a[i]);
    }
    printf("\n");
}

int main() {
    int a[N] = {1, 2, 3, 4, 5};
    printA(a, N);
    printA(&a[2], 2);

    return 0;
}
```

passparambyaddr.c

L'opérateur « & » permet de trouver *l'adresse* d'une variable (sa position en mémoire vive).

Qu'est-ce qui sera affiché dans le terminal?

Les affirmations avec `assert()`

- ❑ Vérification d'une condition. Si c'est faux, affiche un message d'erreur et termine le processus.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h> /* pour memset (!) */
#include <assert.h> /* pour assert */

#define N 5

int main() {
    int a[N] = {1, 2, 3, 4, 5};

    memset(a, 0, N*sizeof(a[0]));
    assert(a[0] == 0);
    memset(a, 1, N*sizeof(int));
    assert(a[0] == 1);

    return 0;
}
```

assert.c

```
pgiard@etsgilen:/tmp/ELE216$ ./a.out
a.out: assert.c:14: main: Assertion `a[0] == 1' failed.
Abandon
```

Remarque:

Typiquement on utilise les `assert()` durant le développement et on les retire lorsqu'on livre une application

Erreur fréquente: Qu'est-ce qu'une erreur de segmentation?

- ❑ C'est un processus qui tente d'accéder un emplacement mémoire qui est en dehors de la plage qui lui a été allouée.

Fichier exemple/segfault.c

```
1  #include <stdlib.h>
2
3  int main(void) {
4      int *a = NULL;
5      *a = 10;
6      return 0;
7  }
8
```

- ❑ Le noyau interrompt le processus

Exercice1

Modifiez ce programme de la façon suivante:

- ☐ Créez 1 nouveau bloc de mémoire de 5 entiers avec malloc().
- ☐ Placez l'adresse de ce bloc dans ptr2.
- ☐ Utilisez memcpy() pour copier les valeurs 5 à 9 du bloc ptr1 vers le bloc ptr2.
- ☐ Utilisez assert() pour vérifier que le malloc() est OK.
- ☐ Imprimer les valeurs pour vérifier le tout.

Fichier exercices/exercice1.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <assert.h>
4
5  int main() {
6      int i;
7      int* ptr1;
8      int* ptr2;
9
10     ptr1 = (int *) malloc(10 * sizeof(int));
11     assert(ptr1);
12     for (int i = 0; i < 10; i++) ptr1[i] = i;
13
14     /* À compléter*/
15
16     return EXIT_SUCCESS;
17 }
18
```

DÉVERMINAGE

Une méthodologie et des outils

Le déverminage (debugging)

- ❑ C'est un processus méthodologique qui consiste à trouver et éliminer les défauts dans un programme, afin que celui-ci ait le comportement attendu.
- ❑ Il y a un bug lorsque:
 - ❖ Le programme échoue un test unitaire
 - ❖ Le programme ne donne pas le résultat attendu
 - ❖ Le programme s'interrompt anormalement, e.g., erreur de segmentation
- ❑ MAIS ce n'est pas parce que ces symptômes ne se manifestent pas qu'il n'y a aucun bug!

Le déverminage : méthodologie

- ❑ Trouver un cas qui permet de répéter la ou les anomalies
- ❑ Bien examiner les résultats obtenus
- ❑ Analyser le code fautif avec des outils:
 - ❖ Ajouter des printf() dans le code
 - ❖ Utiliser un dévermineur (e.g., gdb) pour exécuter le code pas à pas
 - ❖ Utiliser Valgrind pour vérifier l'état de la mémoire dynamique (malloc).
- ❑ Une fois un problème trouvé, corrigez-le et vérifiez que l'anomalie est bien réglée.
- ❑ Si pertinent, ajouter un test unitaire couvrant ce cas de figure

Le déverminage : conseils

- ❑ Le déverminage est un travail de détective !
 - ❖ Essayez le programme avec différentes entrées, et analysez bien les résultats.
 - ❖ Ne jamais utiliser la méthode « essais et erreurs », ça ne marche pas.
 - ❖ Revenez toujours au code original si un changement ne fonctionne pas.
 - ❖ Soyez patient et méthodique.

Déverminage – circonscrire le problème

- ❑ Vous avez trouvé une anomalie:
 - ❖ Trouvez comment la reproduire
 - ❖ Circonscrivez le problème en créant une suite d'opérations minimale permettant de la reproduire
 - ❖ Exécutez le programme dans le dévermineur:
 - Utilisez des point d'arrêt (*breakpoints*)
 - Inspectez l'état des variables
 - ❖ Ou
 - Ajoutez des `printf()` graduellement dans le code

Déverminage – C'est l'impasse, que faire?

- ❑ Demandez l'aide d'un ou d'une collègue (revue de code!)
 - ❖ Exprimer le problème aide à le comprendre, et souvent à le résoudre par nous-mêmes!
 - ❖ Parfois l'erreur est trop évidente
 - Condition inversée
 - Une itération de trop dans une boucle
 - Manque une itération dans la boucle
 - ...
 - ❖ La revue par un pair est un moyen très efficace et peu coûteux

Déverminage avec GDB

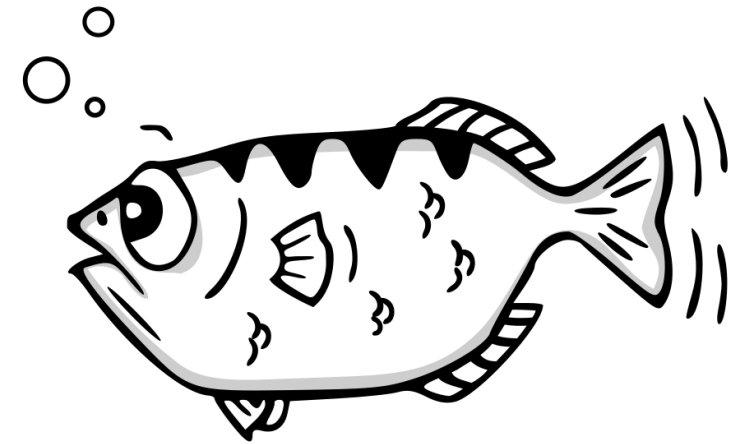
Que fait le dévermineur?

- ☐ Il exécute des programmes
 - ☐ Il permet d'arrêter un programme à un endroit spécifique
 - ☐ Il permet d'arrêter un programme lorsqu'une condition est remplie
 - ☐ Il permet d'inspecter la valeur des variables en cours d'exécution
 - ☐ Il permet d'exécuter le programme pas à pas
-
- ☐ Pour utiliser le dévermineur et s'y retrouver, il faut compiler votre programme avec l'option `-g` et on recommande d'aussi inclure `-O0`, donc:

```
gcc -g -O0 mon_prog.c
```

Déverminage avec GNU gdb

- ❑ On peut utiliser le dévermineur GDB à partir d'un IDE tel que Visual Studio Code.
- ❑ Mais il faut se rappeler que c'est toujours GDB qui fait le travail.



By Original: Jamie Guinan Vectorization: Andreas Arnez -
<https://www.gnu.org/software/gdb/mascot/>, CC BY-SA 3.0 us,
<https://commons.wikimedia.org/w/index.php?curid=107050873>

Déverminage avec GNU gdb à la ligne de commande

Lancer gdb: **`gdb`** `./mon_prog`

Fermer gdb: **`quit`**

run: exécute le programme

quit: quitte la session gdb

break: insère un point d'arrêt:

- **break** <nom de fonction>
- **break** <numéro de ligne>
- **break** <nom de fichier>:<numéro de ligne>
- **break** <condition>

watch <var>: stoppe l'exécution lorsque la variable est modifiée

delete: retire un ou plusieurs points d'arrêt

continue: poursuit l'exécution

step: avance l'exécution pas à pas en entrant dans les fonctions

next: avance l'exécution pas à pas SANS entrer dans les fonctions

print: afficher l'état de la mémoire, c'est une commande d'inspection TRÈS puissante, e.g.,

- **print** <variable>
- **print/x** <variable>
- **print/x** (<cast>)<variable>

Utiliser GNU gdb à la ligne de commande

- Les noms de commande peuvent être tronqués si l'abréviation n'a pas d'ambiguïté, e.g.,
 - **r** pour **run**
 - **s** pour **step**
 - **n** pour **next**
 - **c** pour **continue**
- Utilisation de `TAB` pour compléter les noms de commandes.
- Obtenir de l'aide:
 - help** (ou bien **h**): liste toutes les classes de commandes.
 - h *command***: affiche une courte description de la commande

Analyse de “core dump” avec GDB

1. Compiler avec la table de symboles

```
gcc -g -o segfault segfault.c
```

2. Activer la génération de *core dumps*

```
ulimit -c unlimited
```

3. Exécuter notre application fautive pour qu'elle génère un “*core dump*”

```
./segfault
```

Le “core dump” est créé dans le répertoire **/var/lib/apport/coredump**

4. Démarrer gdb et charger le *core dump*

```
gdb ./segfault -c <nom_du_fichier_core>
```

5. Demander à gdb d'aller à la ligne qui a généré l'erreur de segmentation

```
bt
```

Valgrind détecte les erreurs de gestion de mémoire

❑ Reprenons un exemple

losttouch.c

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int* ptr;
    ptr = (int*) malloc(5 * sizeof(int));
    printf("Adresse de ptr: %p\n", (void *)ptr);

    // La prochaine ligne de code est une erreur.
    // Elle écrase la valeur de ptr ce qui rend le premier bloc
    // de 5 entiers inaccessible et impossible à libérer avec free()

    ptr = (int*) malloc(7 * sizeof(int));
    printf("Adresse de ptr: %p\n", (void *)ptr);

    // seul le bloc de 7 entiers est libéré
    free(ptr);

    // le bloc de 5 entiers ne peut pas être
    // libéré car on ne connaît plus son adresse.
    return 0;
}
```

```
usager@ele216-vb:~/Documents/ELE216/cours03-ex/exemples$ valgrind ./losttouch
==9428== Memcheck, a memory error detector
==9428== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==9428== Using Valgrind-3.13.0 and LibVEX; rerun with -h for copyright info
==9428== Command: ./losttouch
==9428==
Adresse de ptr: 0x522f040
Adresse de ptr: 0x522f4e0
==9428==
==9428== HEAP SUMMARY:
==9428==   in use at exit: 20 bytes in 1 blocks
==9428==   total heap usage: 3 allocs, 2 frees, 1,072 bytes allocated
==9428==
==9428== LEAK SUMMARY:
==9428==   definitely lost: 20 bytes in 1 blocks
==9428==   indirectly lost: 0 bytes in 0 blocks
==9428==   possibly lost: 0 bytes in 0 blocks
==9428==   still reachable: 0 bytes in 0 blocks
==9428==   suppressed: 0 bytes in 0 blocks
==9428== Rerun with --leak-check=full to see details of leaked memory
==9428==
==9428== For counts of detected and suppressed errors, rerun with: -v
==9428== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Comment ça, 20 octets?

Valgrind c'est aussi...

❑ Détecter l'utilisation de variables non initialisées

```
#include <stdio.h>
```

```
int main() {  
    int x;  
    printf("Utilisation avant l'initialisation: %d\n", x);  
}
```

```
Use of uninitialised value of size 8  
at 0x48C4FAB: _itoa_word (_itoa.c:179)  
by 0x48DE843: __vfprintf_internal (vfprintf-internal.c:1687)  
by 0x48CAE5A: printf (printf.c:33)  
by 0x109152: main (dbg1.c:5)
```

❑ Détecter la libération illégale de mémoire

```
#include <stdlib.h>
```

```
int main() {  
    int x = 6;  
    int *y = &x;  
    free(y);  
}
```

```
Invalid free() / delete / delete[] / realloc()  
at 0x48399AB: free (vg_replace_malloc.c:538)  
by 0x109157: main (dbg2.c:6)  
Address 0x1ffefffff64 is on thread 1's stack  
in frame #1, created by main (dbg2.c:3)
```

Exercices avec Valgrind

- ❑ Compilez le programme test_valgrind.c avec gcc, en activant la table de symboles (-g)
 - ❖ \$ gcc -g -o test_valgrind test_valgrind.c
- ❑ Exécutez le programme avec valgrind
 - ❖ \$ valgrind ./test_valgrind
- ❑ Modifiez le programme afin d'éliminer les anomalies trouvées par valgrind

```
1  #include <stdlib.h>
2
3  int main() {
4
5      // 1- erreur de segmentation
6      int *p;
7      p = NULL;
8      // *p = 0;
9
10     // 2- accès à l'extérieur d'un bloc
11     p = (int*)malloc(4 * sizeof(int));
12     p[5] = 0;
13
14     // 3- utiliser une variable non-initialisée
15     int i, j;
16     j = i + 3;
17
18     // 4- utiliser une zone mémoire non-initialisée
19     j = p[0];
20
21     // 5- écraser le pointeur d'un bloc
22     p = (int*) malloc(7 * sizeof(int));
23
24     // 6- accès à une zone désalouée
25     free(p);
26     p[0] = 0;
27
28     return 0;
29 }
30
```

Exercices `todbg1.c`, `todbg2.c`, `todbg3.c`, `todbg4.c` `todbg5.c`

- ❑ Utilisez les méthodes de votre choix (e.g., des `printf`, `gdb`, `valgrind`, ...) afin de trouver le ou les problèmes dans:
 - ❖ `todbg1.c`,
 - ❖ `todbg2.c`,
 - ❖ `todbg3.c`,
 - ❖ `todbg4.c`,
 - ❖ `todbg5.c`
- ❑ Rappel: N'hésitez pas à consulter les manpages de fonctions que vous ne connaissez pas.

Exercice ultime

- ❑ Implémentez votre propre liste chaînée
- ❑ Chaque élément de votre liste chaînée doit contenir un entier signé
- ❑ Au minimum, vous devez avoir...
 - ❖ Une fonction pour ajouter un élément à la fin de la liste
 - ❖ Une fonction pour retirer le dernier élément de la liste
 - ❖ Une fonction qui itère au travers de la liste, de la tête à la queue, et qui affiche la valeur de l'entier de l'élément
- ❑ N'utilisez pas l'implémentation de quelqu'un d'autre!
- ❑ Essayez de vous en tenir aux « man pages » pour l'aide sur les fonctions
- ❑ Utilisez les méthodes vues aujourd'hui pour vous assurer que votre code fonctionne correctement